Intitulé du cours: Technique des applications web

Framework

Model Class

Chapitre 3: Migrations et Base de Données

Filière: 2éme année – bts web

Enseignante: Abid Arij

Année universitaire 2025/2026

Objectifs du module

À la fin de ce module, l'étudiant sera capable de **concevoir, créer, manipuler et gérer une base de données relationnelle** dans une application Django en utilisant **l'ORM**, les **modèles**, les **relations** et les **migrations**, sans écrire directement du SQL.

- Configurer une base de données dans un projet Django
- Créer et gérer des modèles Django
- Mettre en œuvre les relations entre modèles
- Utiliser les migrations Django
- Manipuler les données avec l'ORM Django (CRUD)
- Garantir la cohérence et la sécurité des données

Définition d'une base de données django

La BD django utilisée par défaut est :

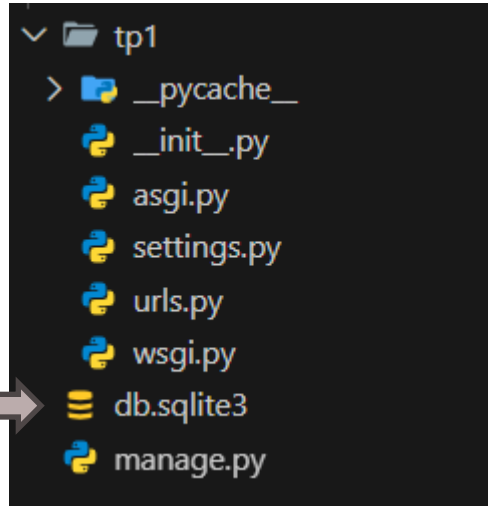
✓ **SQLite (sqlite3)**

Pourquoi ?

- elle est **légère**
- **déjà intégrée**
- zéro configuration
- idéale pour **débutante** et petits projets

▶ Le fichier de BD est :

monprojet/db.sqlite3



Les types de bases de données supportés par Django :

On peut changer pour d'autres BD comme :

- MySQL
- PostgreSQL
- Oracle
- SQL Server

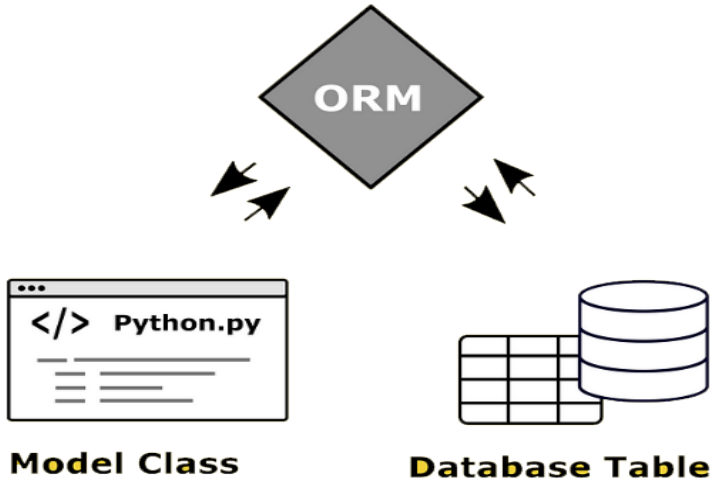
SQLite vs MySQL Workbench

Technologie	Ce que c'est	Quand l'utiliser
SQLite	Fichier simple .sqlite3	Débutantes, tests, petits projets
MySQL Workbench + MySQL	Serveur de base de données puissant, interface graphique Workbench	Applications professionnelles, gros volume de données

Qu'est-ce qu'une base de données dans Django ?

Django utilise une base de données pour **stocker les données de ton application**.
Exemples de données stockées :

- utilisateurs
- produits
- commandes
- articles
- commentaires
- notes d'étudiants…



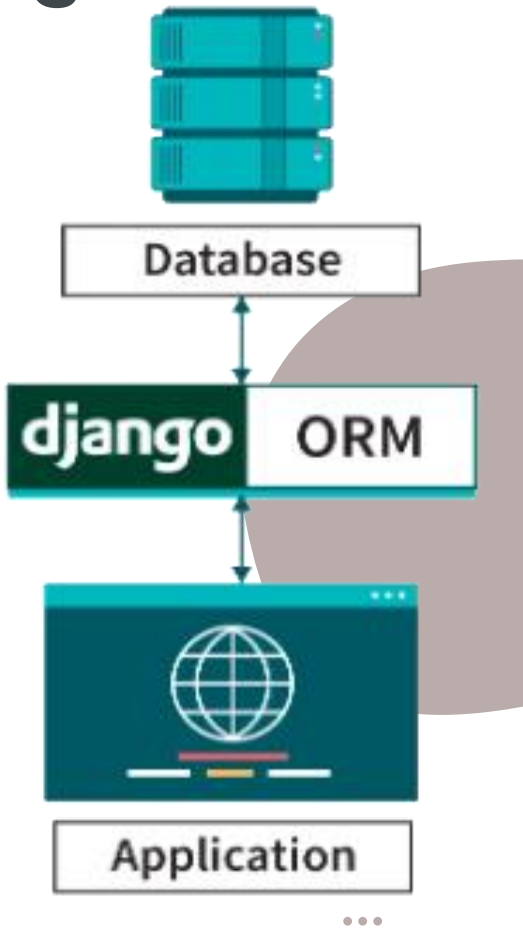
Django permet de **gérer la base automatiquement grâce au système :**

ORM = Object Relational Mapping

Qu'est-ce qu'une base de données dans Django ?

En Django, **une base de données (BD)** est l'endroit où **l'application stocke ses informations** :
utilisateurs, produits, articles, notes, photos, réservations…
Django utilise un **ORM (Object-Relational Mapping)** pour communiquer avec la BD.

Dans Django, tu n'écris jamais directement des requêtes SQL (comme SELECT, INSERT…). À la place, Django utilise un **ORM**.



Configuration de la base de données

Fichier settings.py
Paramètres principaux : ENGINE, NAME, USER, PASSWORD, HOST, PORT
Exemple pour SQLite et MySQL :

```
DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.mysql',
        'NAME': 'tp_django',
        'USER': 'root',
        'PASSWORD': 'root',
        'HOST': '127.0.0.1',
        'PORT': '3306',
        'OPTIONS': {
            'charset': 'utf8mb4',
        }
    }
}
```

Les modèles Django (models.py)

Un **model** = une classe Python qui représente une **table** dans la base de données.

Django Models

- Les classes Models sont mappés à des tables
- Les attributs sont mappés à des colonnes
- SQL est généré
- Create/Update tables (migrations)

  Insert/Update/Delete lignes (admin)

Exemple :
une classe Client → une table client
une classe Produit → une table produit
une classe Article → une table article
Chaque attribut = une colonne dans la table.

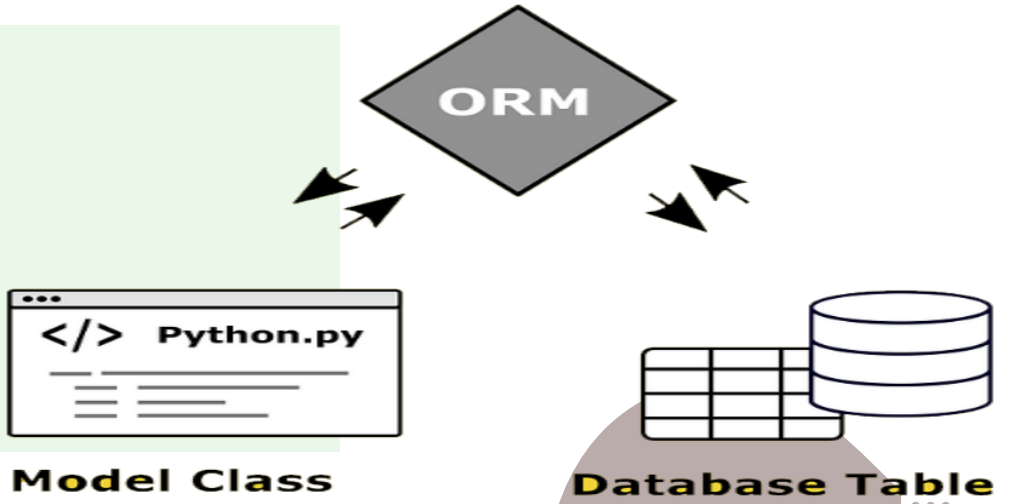
Les modèles Django (models.py)

Un **model** est une **classe Python** qui représente une **table** dans la base de données.
Un champ de la classe → **une colonne dans la table**.

Exemple:

- Si tu écris une classe Student, Django crée une table student_student dans la base de données.

```
from django.db import models

class Student(models.Model):
    name = models.CharField(max_length=100)
    age = models.IntegerField()
```



Chaque attribut = une colonne dans la table.
Chaque instance (objet) = une ligne dans la table.

Models

In the Code

```
Class Article():
    title = models.CharField()
    body = models.TextField()
```

In the Database

Articles

id	title	body
1	Django	les bases de Django
2	HTML	les bases de HTML
3	CSS	les bases de CSS

Les champs les plus utilisés dans un Model Django

Django	Type SQL	Explication
CharField	VARCHAR	Texte court
TextField	TEXT	Long texte
IntegerField	INT	Nombre entier
FloatField	FLOAT	Nombre décimal
BooleanField	BOOLEAN	Vrai / Faux
DateTimeField	DATETIME	Date + heure
FileField	FILE	Fichier
ImageField	IMAGE	Image
ForeignKey	FK	Relation 1 → N
ManyToManyField	M2M	Relation N ↔ N

```
from django.db import models

class Article(models.Model):
    titre = models.CharField(max_length=100)
    contenu = models.TextField()
    date_creation = models.DateTimeField(auto_now_add=True)
```

Classe / attribut	Table / colonne SQL
Article	table "article"
titre	colonne VARCHAR(100)
contenu	colonne TEXT
date_creation	colonne DATETIME

1. Django crée automatiquement la table SQL → pas besoin de langage SQL.

Les modèles : la base de la BD en Django

```
from django.db import models

class Etudiant(models.Model):
    nom = models.CharField(max_length=100)
    prenom = models.CharField(max_length=100)
    age = models.IntegerField()
    email = models.EmailField()
```

Django va créer automatiquement une table SQL équivalente à :

```
CREATE TABLE app_etudiant (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    nom VARCHAR(100),
    prenom VARCHAR(100),
    age INTEGER,
    email VARCHAR(254)
);
```

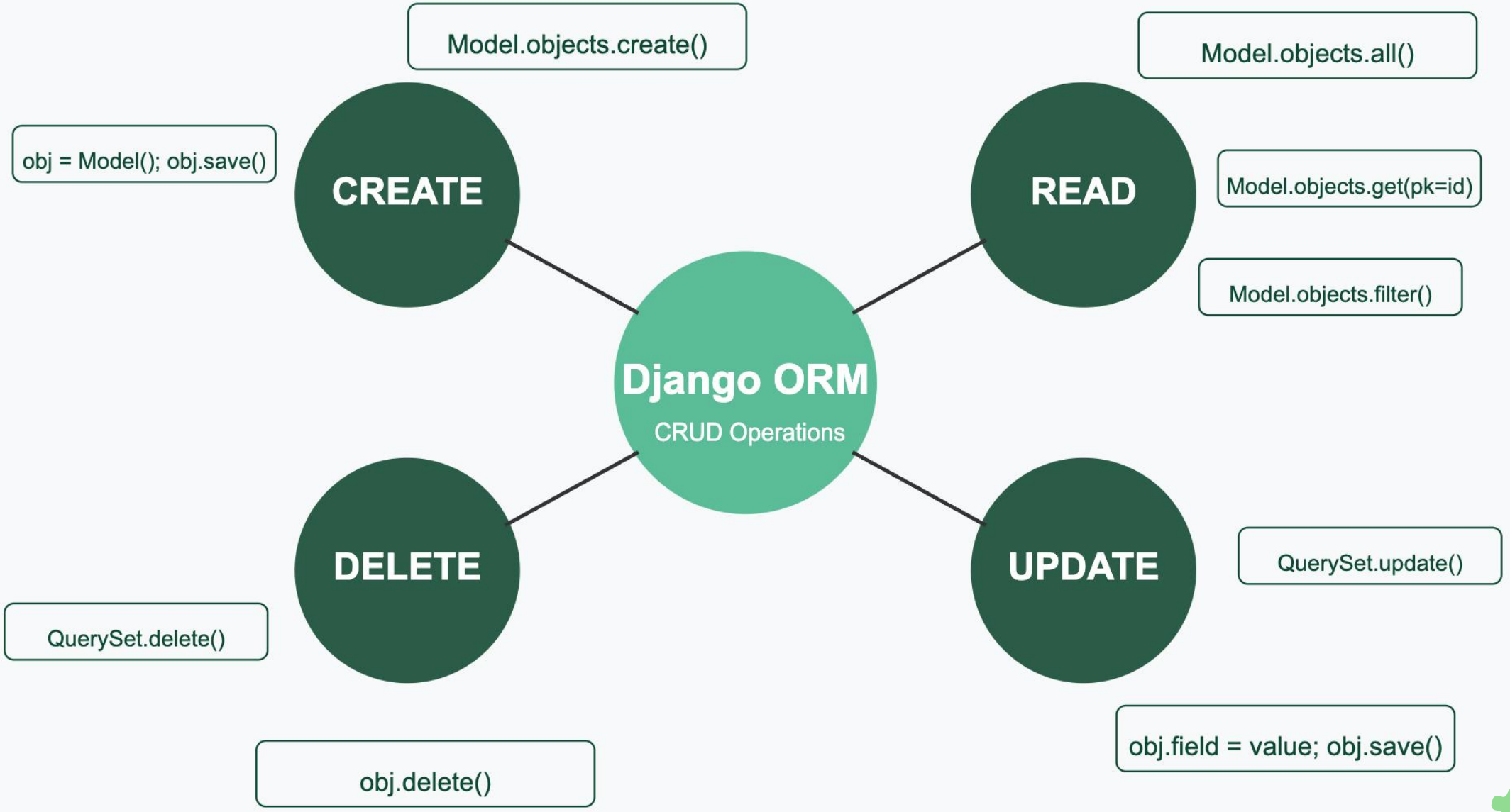
L'ORM Django (Object Relational Mapping)

L'ORM est un système qui permet de **manipuler la base de données sans écrire du SQL**.
C'est un outil qui **transforme vos objets Python en tables dans la base de données**.

L'ORM te permet de manipuler la base avec du Python, pas avec du SQL.
Au lieu d'écrire : SELECT * FROM produit;
Tu écris : Produit.objects.all()

- Django traduit ton code Python en SQL automatiquement.

Les opérations CRUD en Django



Le rôle de l'ORM Django

L'ORM sert de traducteur :

Vous écrivez en Python	Django traduit en SQL	La BD fait
Article.objects.all()	SELECT * FROM article	Récupère les données
Article(titre="Test").save()	INSERT INTO article ...	Ajoute une ligne
Article.objects.filter(id=3)	SELECT * FROM article WHERE id=3	Filtre

Exemple :

Vous créez un objet Utilisateur en Python.
Avec ORM, Django va automatiquement créer la table correspondante dans la base de données.

Exemples :

➤ **Ajouter un étudiant :**

```
Student.objects.create(name="Ali", age=21)
```

➤ **Lire les étudiants :**

```
Student.objects.all()
```

➤ **Filtrer :**

```
Student.objects.filter(age=21)
```

ORM = Django traduit automatiquement ton code Python en SQL.
Tu n'as pas besoin d'écrire :
SELECT * FROM student WHERE age = 21;
Django le fait pour toi !

Manipulation des données via ORM

❖ **CREATE — Ajouter un enregistrement**

```
article = Article.objects.create(
    titre="Mon premier article",
    contenu="Contenu de test"
)
ou :
article = Article(
    titre="Nouveau titre",
    contenu="Texte..."
)
article.save()
```

❖ **DELETE — Supprimer**

```
article = Article.objects.get(id=1)
article.delete()
```

❖ **READ — Lire / récupérer les données**

➤ **Tout afficher**

```
Article.objects.all()
```

➤ **Trouver par ID**

```
Article.objects.get(id=1)
```

➤ **Filtrer**

```
Article.objects.filter(titre="Mon premier article")
```

➤ **Filtrer contenant un mot**

```
Article.objects.filter(titre__contains="article")
```

❖ **UPDATE — Modifier**

```
article = Article.objects.get(id=1)
article.titre = "Titre modifié"
article.save()
```

Tables système créées par Django

- auth_user → utilisateurs
- auth_group → groupes
- django_migrations → historique des migrations
- django_content_type, django_session, django_admin_log

Les relations entre modèles (ORM Relations)

Les relations entre modèles (ORM Relations)

Relations entre modèles

- One to One Relationship
- One to Many Relationships
- Many to Many Relationships

Les relations entre modèles (ORM Relations)

En base de données relationnelle, une **relation** définit comment deux tables sont liées entre elles.

En Django, on définit ces relations dans **models.py** avec des champs spéciaux :

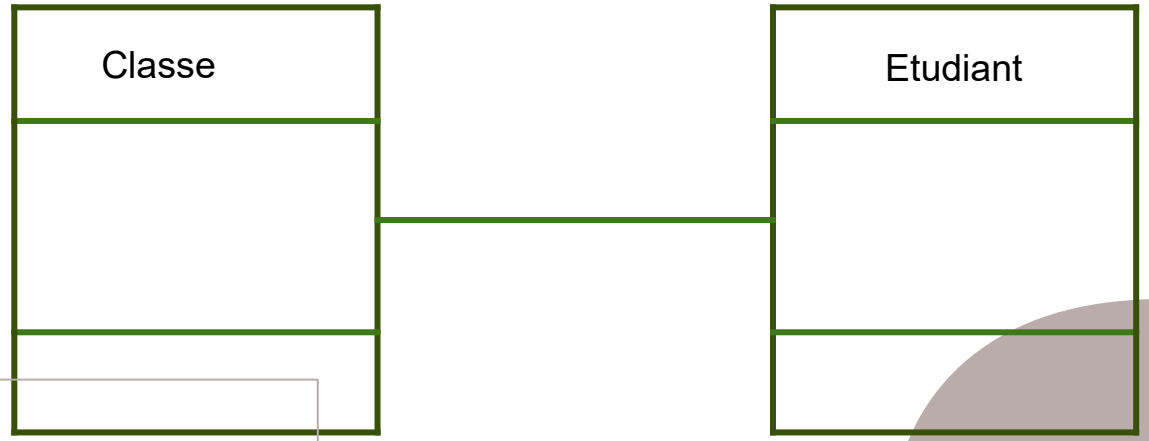
- **Many-to-One (ForeignKey)**
- **Many-to-Many (ManyToManyField)**
- **One-to-One (OneToOneField)**

Chaque relation crée **des clés étrangères** dans la base de données.

Relation One-To-Many (Un-à-Plusieurs)

Un **objet A** peut être lié à **plusieurs objets B**.
Mais un **objet B** n'appartient qu'à **un seul objet A**.
💡 Exemple classique :

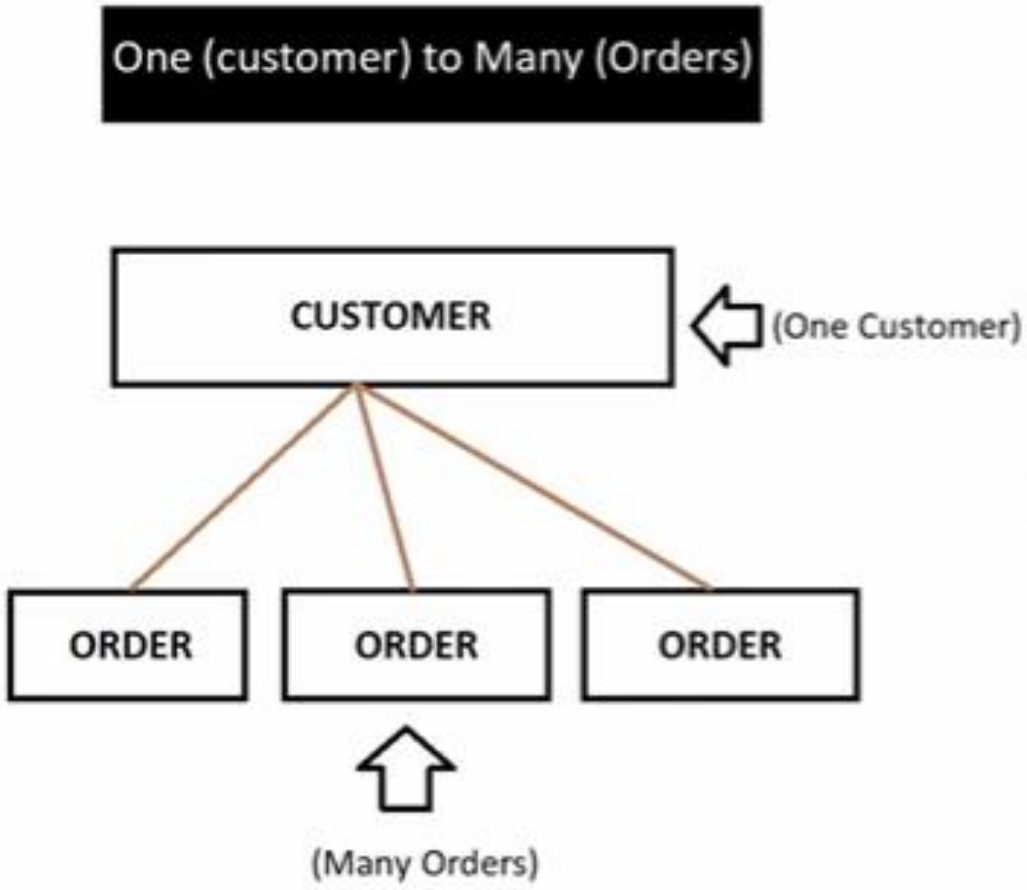
- Une **classe** a **plusieurs étudiants**.
- Un étudiant appartient **à une seule classe**.




```
from django.db import models

class Classe(models.Model):
    nom = models.CharField(max_length=50)
    niveau = models.CharField(max_length=20)


class Etudiant(models.Model):
    nom = models.CharField(max_length=100)
    prenom = models.CharField(max_length=100)
    age = models.IntegerField()
    classe = models.ForeignKey(Classe, on_delete=models.CASCADE)
```

`on_delete=models.CASCADE` → si la classe est supprimée, tous les étudiants liés sont supprimés aussi.

Exemple Relation One-To-Many (Un-à-Plusieurs)



(Customers)

ID	NAME
1	John
2	Tim
3	Erik

(ORDERS)

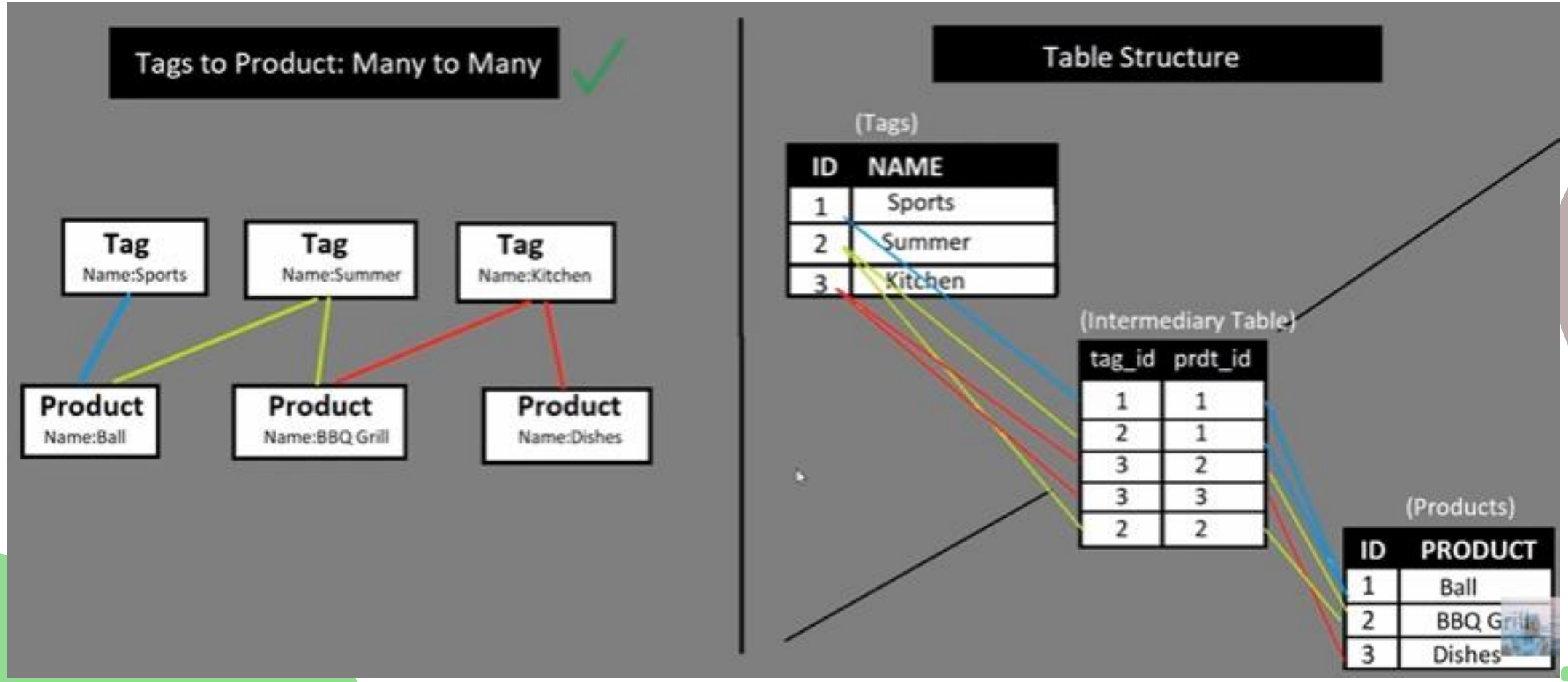
ID	CUS_ID	PRODUCT
1	1	Basketball
2	1	BBQ Grill
3	3	Dishset
4	2	Basketball
5	2	Couch
6	3	BBQ Grill

Relation Many-To-Many (Plusieurs-à-Plusieurs)

Plusieurs entités A peuvent être liées à plusieurs entités B.
Exemple : **Un étudiant peut suivre plusieurs cours**, et **un cours peut avoir plusieurs étudiants**.

```
class Cours(models.Model):
    nom = models.CharField(max_length=50)

class Etudiant(models.Model):
    nom = models.CharField(max_length=100)
    cours = models.ManyToManyField(Cours)
```

Exemple Relation Many-To-Many (Plusieurs-à-Plusieurs)



Relation One-To-One (Un-à-Un)

Une ligne de A correspond **exactement** à une ligne de B.

- Exemple : chaque **profil utilisateur** correspond à **un seul utilisateur**.

```
from django.contrib.auth.models import User
class Profil(models.Model):
    user = models.OneToOneField(User, on_delete=models.CASCADE)
    date_naissance = models.DateField()
    adresse = models.CharField(max_length=200)

    def __str__(self):
        return self.user.username
```

Résumé des relations avec schéma

Relation	Exemple concret	Champ Django	Cardinalité
One-to-Many	Professeur → Cours	ForeignKey	1→N
Many-to-Many	Étudiant ↔ Cours	ManyToManyField	N↔N
One-to-One	User ↔ Profil	OneToOneField	1→1

Comment fonctionne l'ORM en interne ?

- **Étape 1 : Tu écris une classe Python (Model)**
  Django convertit ça en SQL (migrations)
- **Étape 2 : Tu utilises des objets Python**
  Django convertit ça en requêtes SQL automatiques

Exemple :
Article.objects.filter(id=1)
Django crée une requête SQL cachée :
SELECT * FROM blog_article WHERE id = 1;
⇨ L'ORM fait **la traduction Python → SQL** automatiquement.

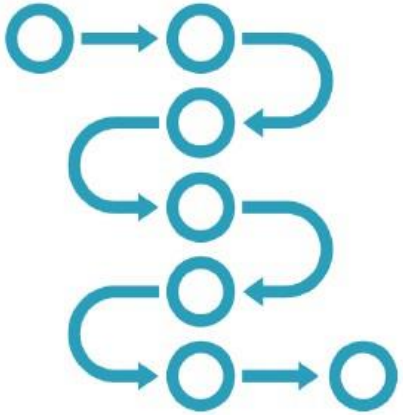
Les migrations Django

Une **migration** est un fichier généré par Django qui permet de **traduire les modèles Python (models.py)** en **tables dans la base de données**.

Models.py ➜ Migration ➜ Base de données

Models

Classes Python
Associées aux tables de
la BD



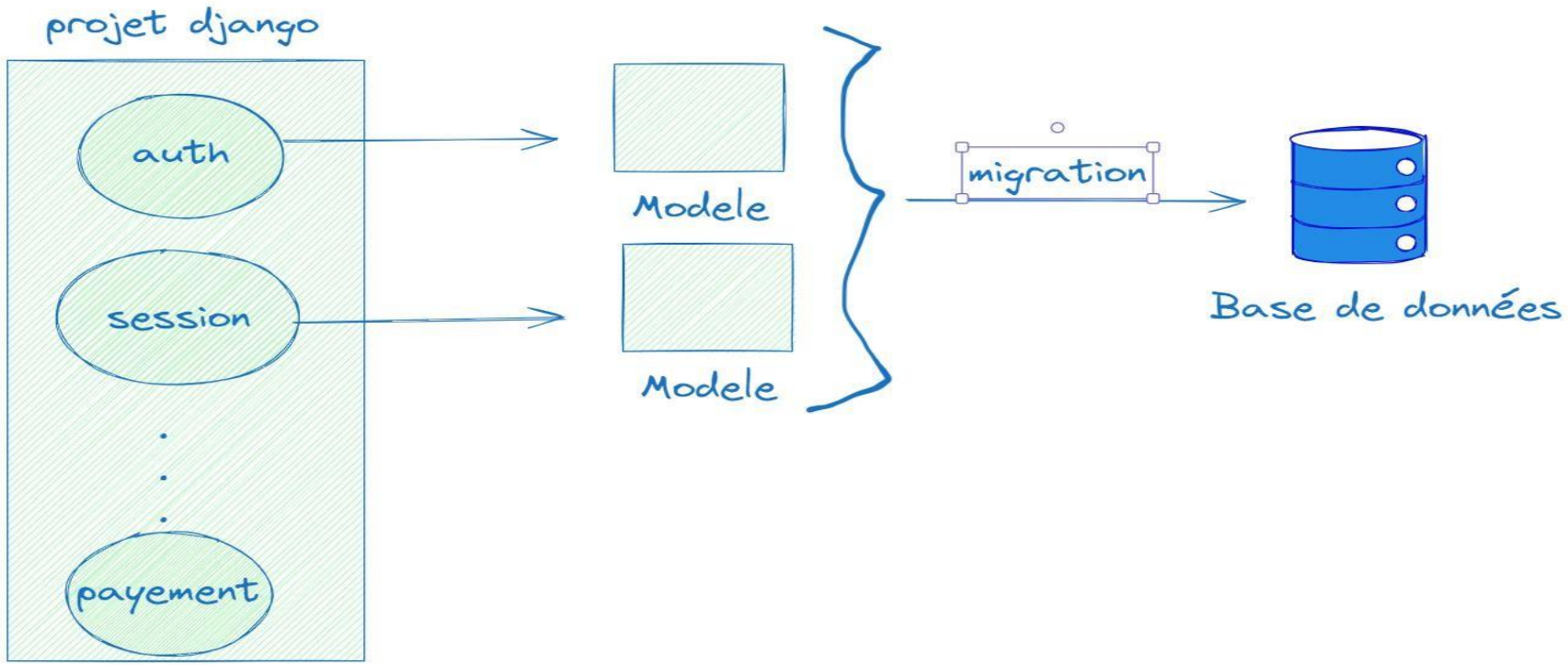
Migrations

Ensemble de scripts
décrivant le schéma de
la BD

Comment Django crée la base de données ?

Django ne crée **pas automatiquement la base de données complète** au niveau du serveur.
Ce qu'il fait :

- Il lit la configuration dans le fichier **settings.py**.
- Il utilise **le moteur de base de données choisi** (SQLite, MySQL, PostgreSQL, Oracle…).
- Il crée les **tables correspondant aux modèles Python (models.py)** via les **migrations**.



Démarche: Models

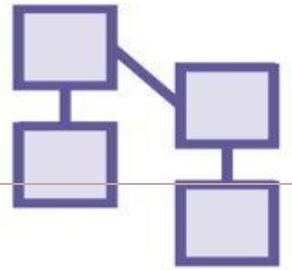
Étape par Étape

- Créer les classes modèles
- Créer les migrations
- appliquer les migrations
- Gérer le modèle avec Admin-interface

Créer le modèle

créer un model dans shop/models.py

Exemple d'un model **Client** :



```
from django.db import models

class Client(models.Model):
    nom = models.CharField(max_length=50)
    email = models.EmailField(unique=True)
    age = models.IntegerField()
    date_creation = models.DateTimeField(auto_now_add=True)

    def __str__(self):
        return self.nom
```

Créer une migration

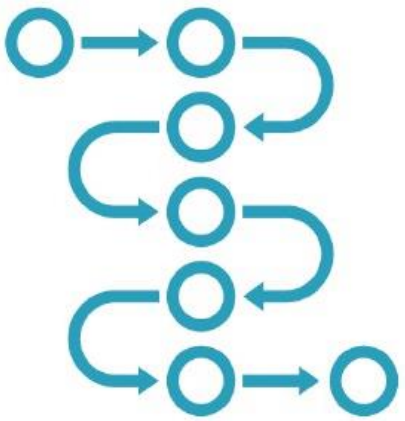
Chaque fois que vous créez ou modifiez un modèle, Django doit créer une migration avec :
python manage.py makemigrations

Ce que fait la commande :

- **Analyse les modèles Django (models.py)**
- **Détecte les changements** par rapport aux migrations déjà existantes
- **Crée des fichiers de migration** (instructions)
- **NE modifie PAS la base de données**

Elle prépare les changements, mais ne les applique pas.

Création d'un fichier dans :
nom_de_l_app/migrations/0001_initial.py



Migrations

Appliquer les migrations initiales

Une migration = un **fichier généré automatiquement** par Django qui convertit le modèle en une **table dans la base de données**.

Exécute les fichiers de migration

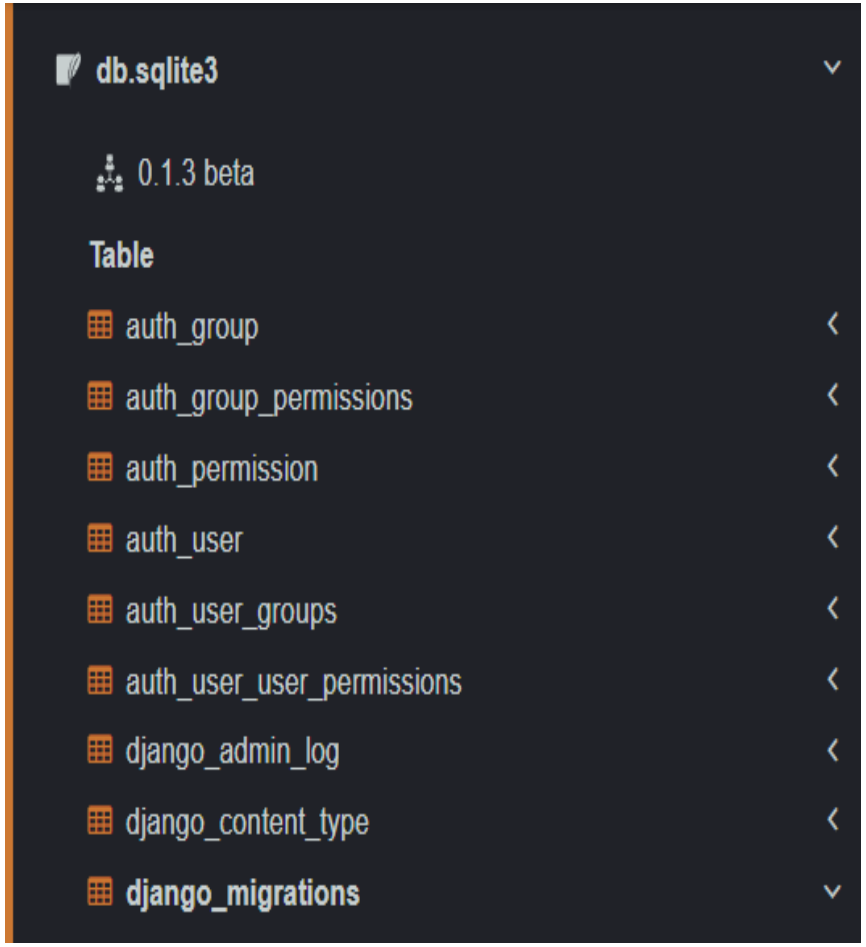
👉 **Crée / modifie les tables dans la base de données**

👉 **Met à jour la table django_migrations**

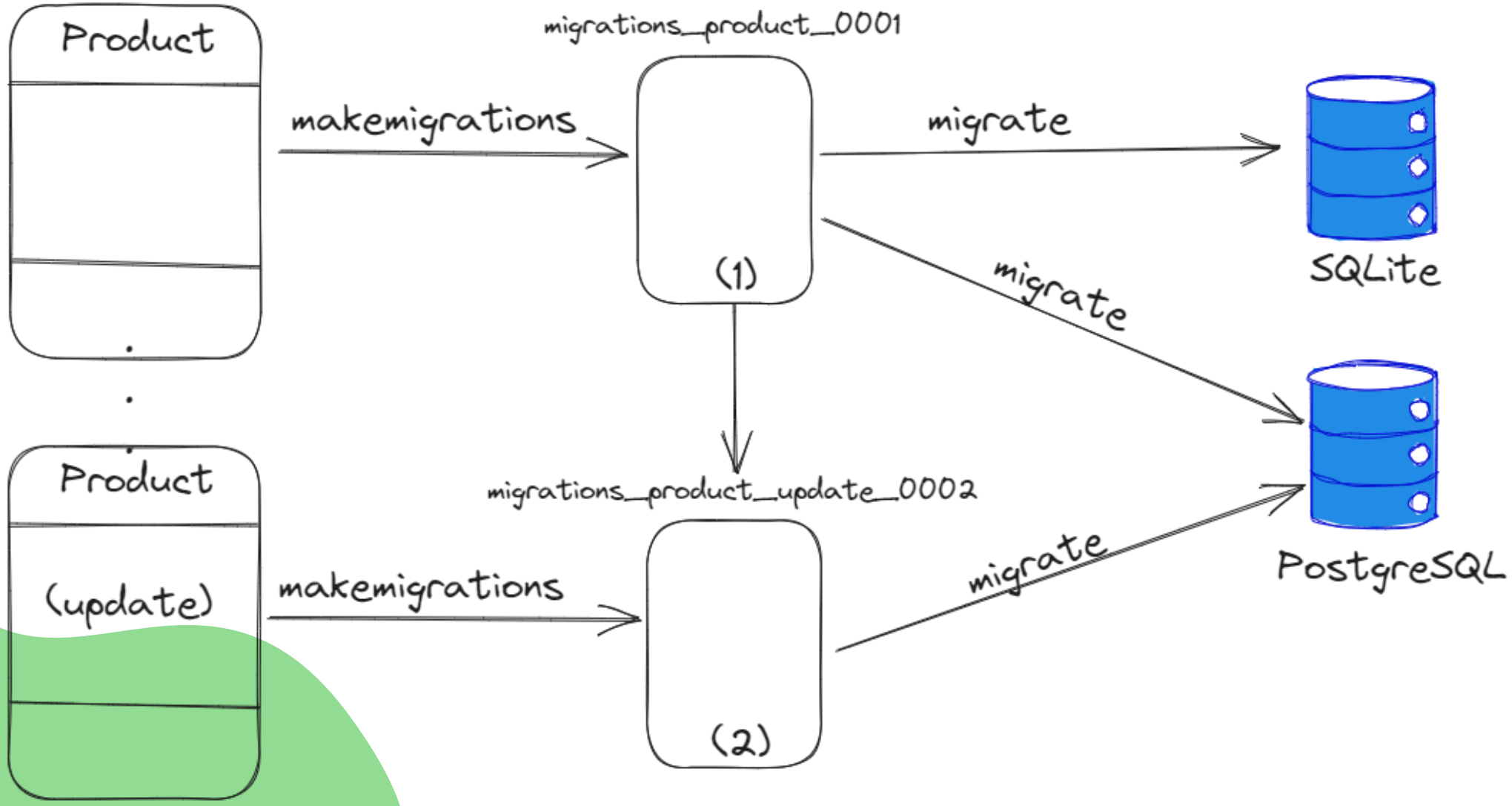
```
# appliquer les migrations

(my_env) $ python manage.py migrate
```

Afficher le schéma de la BD: https://sqliteonline.com/

📌 **Sans migration → pas de table !**



Models et Migrations



Show migrations

```
(.venv) PS C:\Users\asus\tp1_django> python manage.py showmigrations
FirstApp
 [X] 0001_initial
admin
 [X] 0001_initial
 [X] 0002_logentry_remove_auto_add
 [X] 0003_logentry_add_action_flag_choices
auth
 [X] 0001_initial
 [X] 0002_alter_permission_name_max_length
 [X] 0003_alter_user_email_max_length
 [X] 0004_alter_user_username_opts
 [X] 0005_alter_user_last_login_null
 [X] 0006_require_contenttypes_0002
 [X] 0007_alter_validators_add_error_messages
 [X] 0008_alter_user_username_max_length
 [X] 0009_alter_user_last_name_max_length
 [X] 0010_alter_group_name_max_length
 [X] 0011_update_proxy_permissions
 [X] 0012_alter_user_first_name_max_length
contenttypes
 [X] 0001_initial
 [X] 0002_remove_content_type_name
sessions
 [X] 0001_initial
```

Cycle de migration

```
# créer la migration

(my_env) $ python manage.py makemigrations

# afficher SQL de la migration

(my_env) $ python manage.py sqlmigrate meetings 0001

# appliquer la migration

(my_env) $ python manage.py migrate meetings

# Afficher les migrations disponibles
(my_env)  python manage.py showmigrations
```

Relation entre Model → Migration → Base de données

Étape	Explication
1.Tu écris un **Model**	Classe Python = description de ta table
2.Django génère une **migration**	Traduction en instructions pour BD
3.Tu appliques la migration	Django crée/modifie la table dans MySQL

Merci pour votre attention !